

This is CS50

Week 5

Kelly Ding (she/her)

Preceptor

kelly@cs50.harvard.edu

- What are the key **trade-offs** between data structures we should consider in decisions about which to use?
- What are some of the primary operations we should know how to do on a **linked list**?
- How can we build our very own **hash table**?

Part 0

Choosing Data Structures

Imagine you work for a
company that has created a
digital assistant running on
a mobile device.

Customer reports indicate
people have trouble
activating the assistant
with its "wake word."

Your team has been asked to
ensure the voice assistant
can be **awoken with a
greater variety of words.**

What **data structure** would
you propose the team build to
store these words?

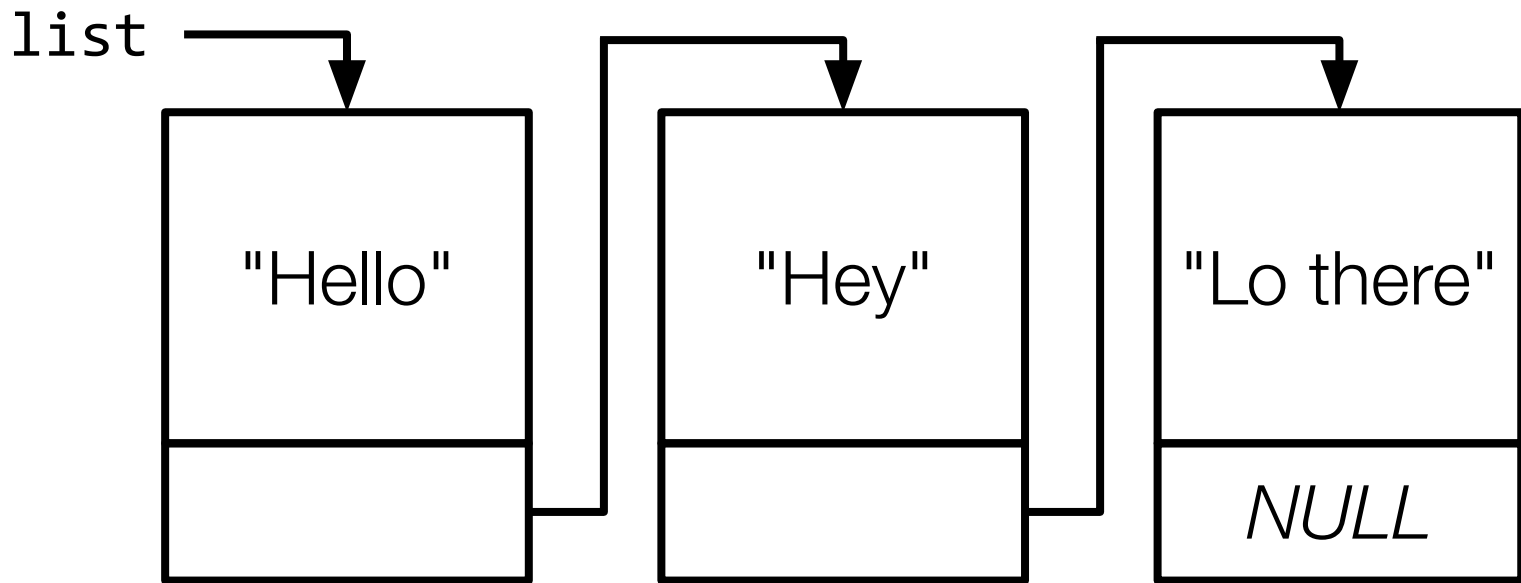
Deletion
Insertion
Search

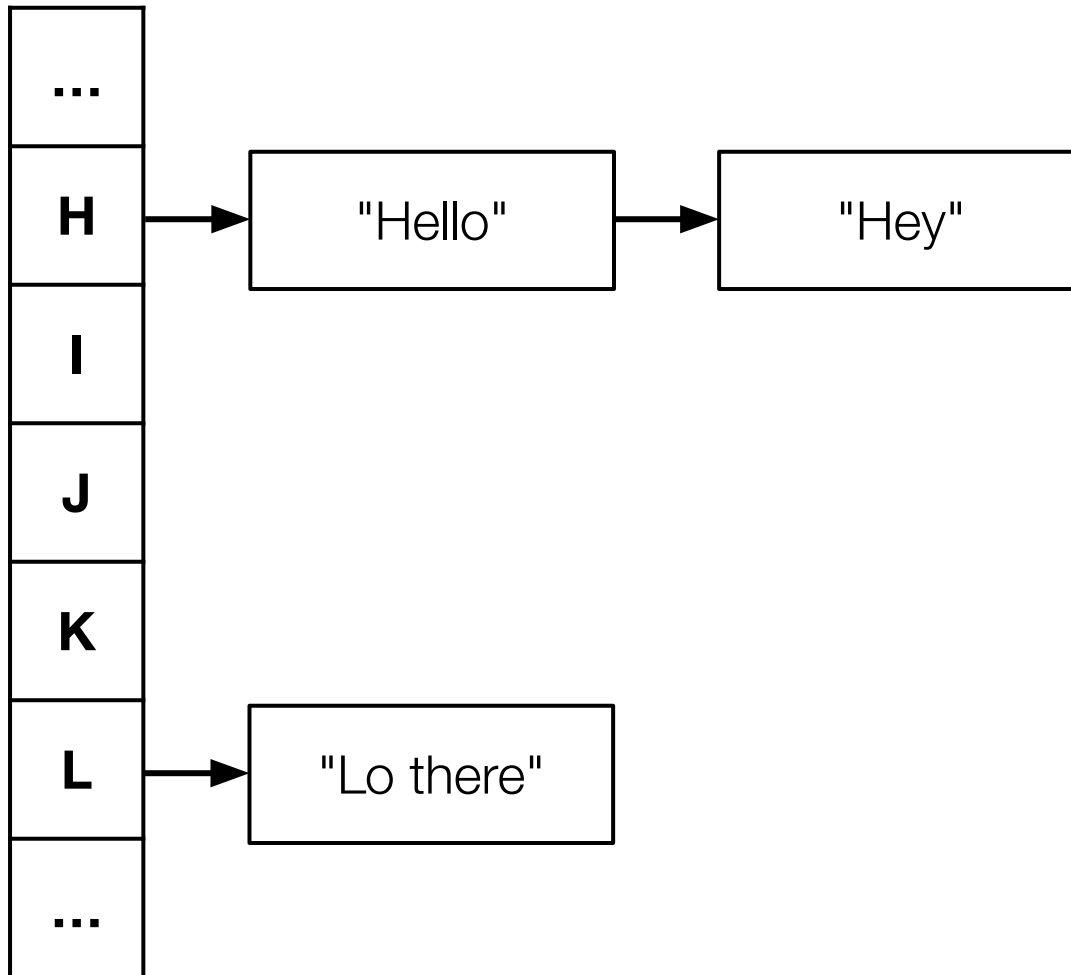
What impact does your prioritization have?

1. Search
2. Insertion
3. Deletion

1. Insertion
2. Search
3. Deletion

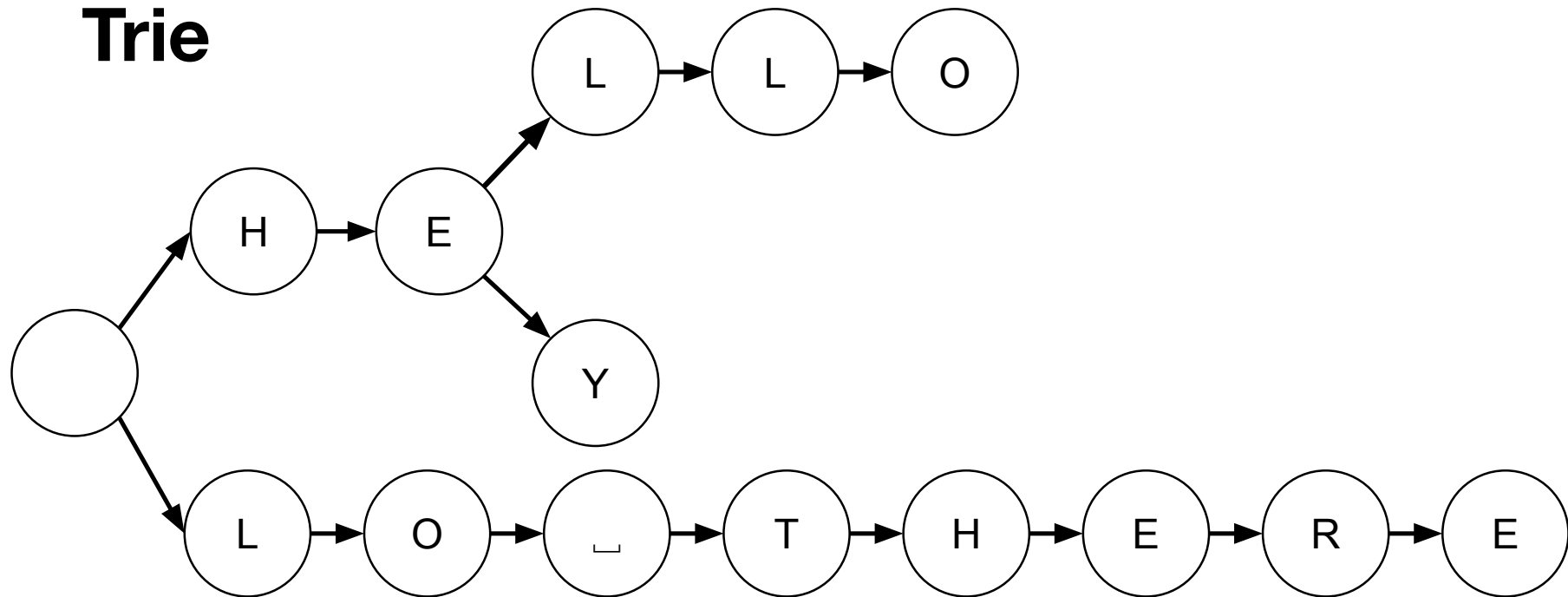
Linked List





Hash Table

Trie



Trade-offs

Big Board speller

Rank	Name	Time	Load	Check	Size	Unload	Memory	Heap	Stack
1	Thomas Ballatore Staff	6.136 s	1.234 s	4.902 s	0.000 s	0.000 s	12.3 kB	4.6 kB	
2	CarterZenke	7.119 s	0.932 s	5.651 s	0.000 s	0.536 s	8.0 MB	8.0 MB	
3	zachatoch1	10.248 s	1.079 s	8.319 s	0.000 s	0.850 s	8.0 MB	8.0 MB	95

Time is a sum of the times required to spell-check `texts/*.txt` using `dictionaries/large`. **Memory** is a measure of maximal heap and stack utilization when spell-checking `texts/holmes.txt` using `dictionaries/large`.

Big Board speller

Rank	Name	Time	Load	Check	Size	Unload	Memory	Heap	St
1	Thomas Ballatore Staff	6.136 s	1.234 s	4.902 s	0.000 s	0.000 s	12.3 kB	4.6 kB	
2	CarterZenke	7.119 s	0.932 s	5.651 s	0.000 s	0.536 s	8.0 MB	8.0 MB	
3	zachatoch1	10.248 s	1.079 s	8.319 s	0.000 s	0.850 s	8.0 MB	8.0 MB	95

Time is a sum of the times required to spell-check `texts/*.txt` using `dictionaries/large`. **Memory** is a measure of maximal heap and stack utilization when spell-checking `texts/holmes.txt` using `dictionaries/large`.

Part 1

Linked List

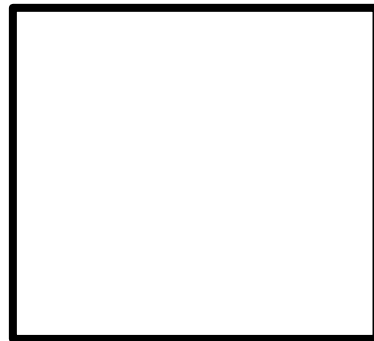
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

node

```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

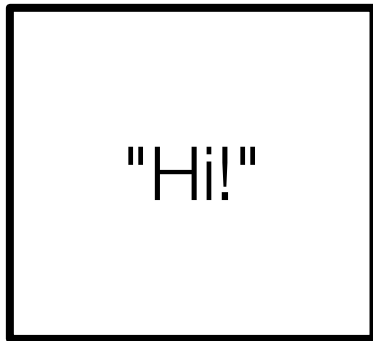
node



phrase

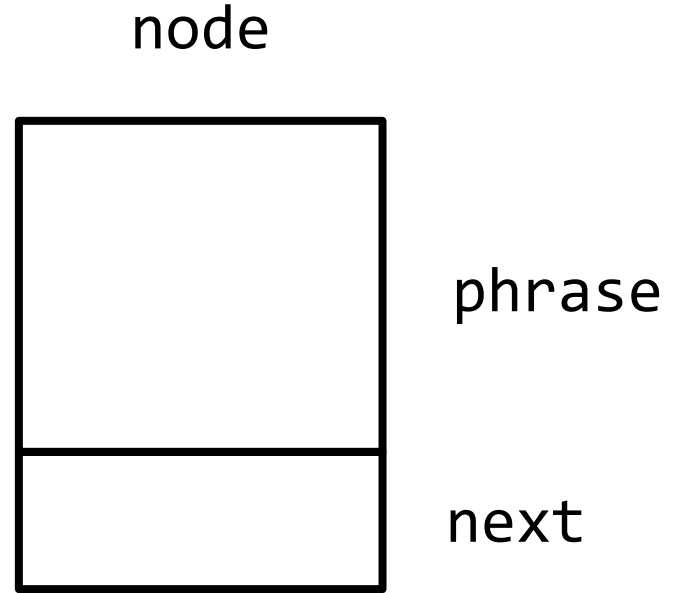
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

node

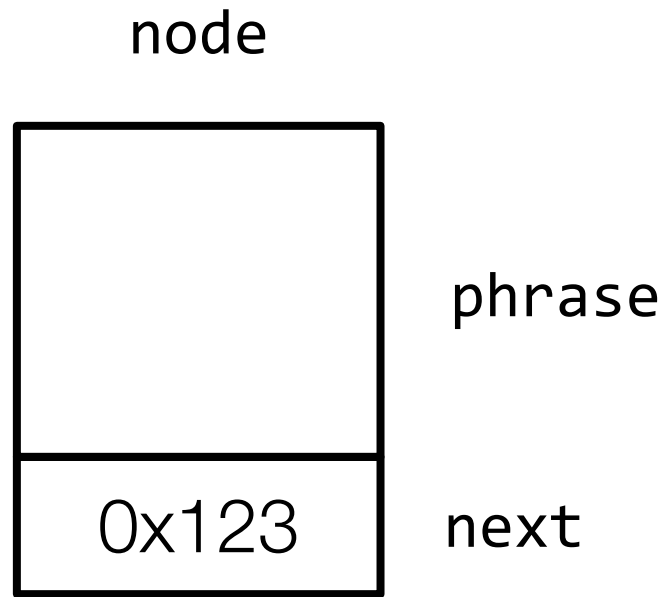


phrase

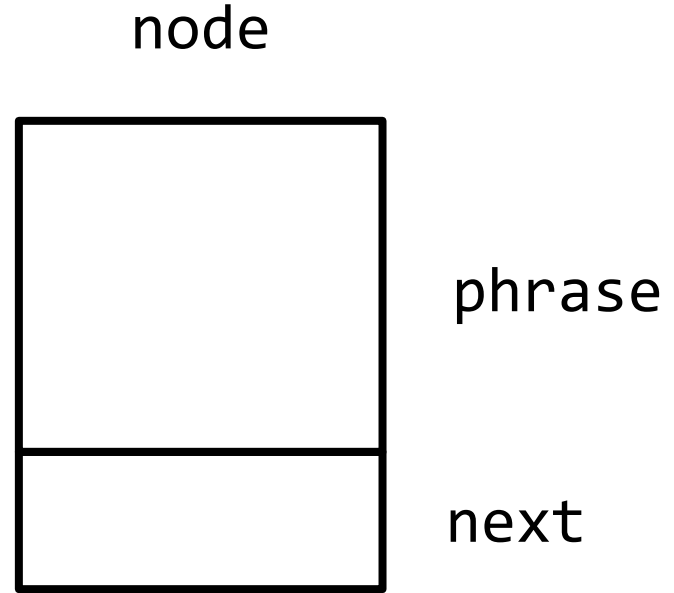
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



Creating a Linked List

```
node *list = NULL;
```

list



NULL

```
node *n = malloc(sizeof(node));
```

list



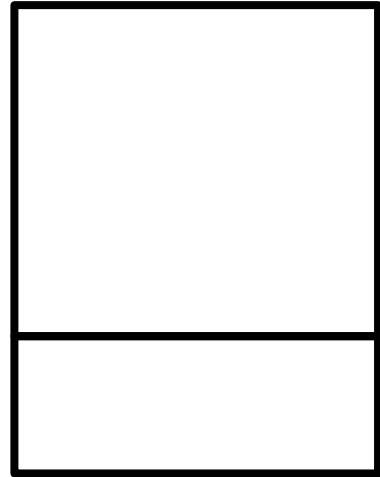
NULL

```
node *n = malloc(sizeof(node));
```

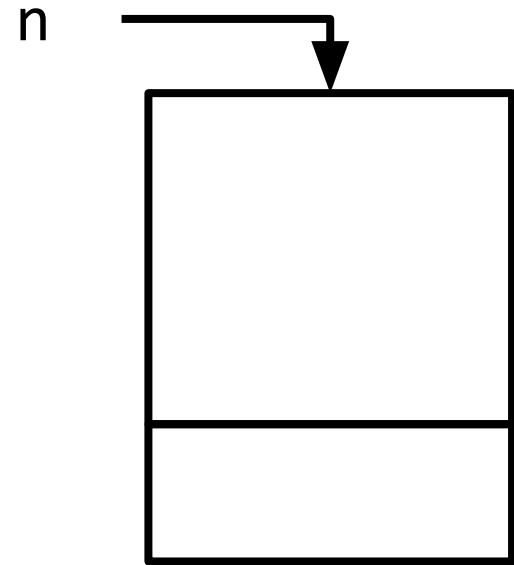
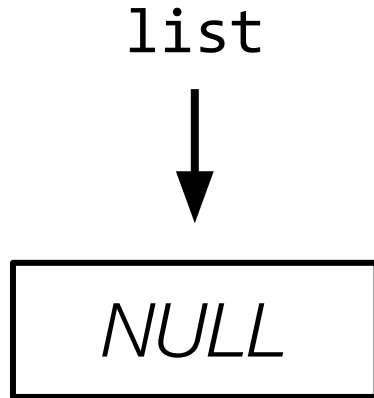
list



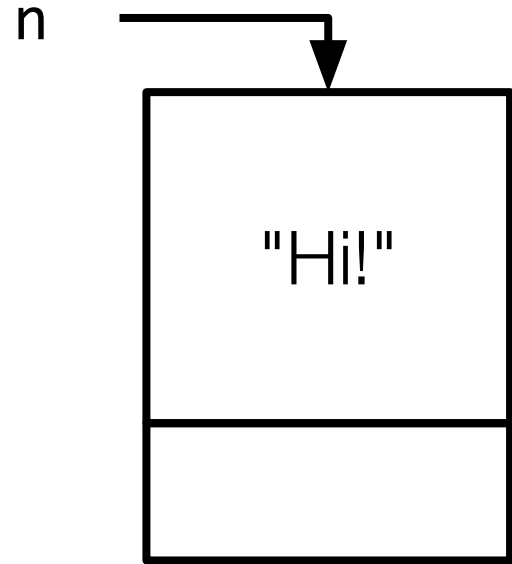
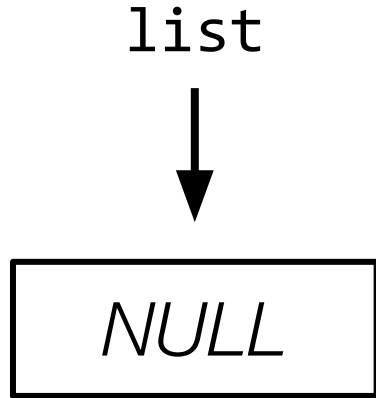
NULL



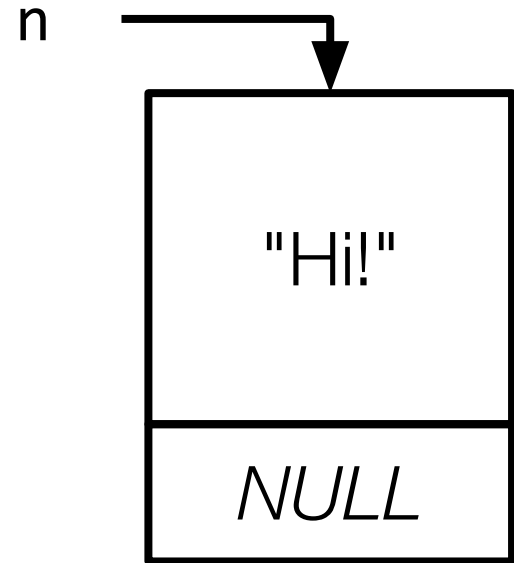
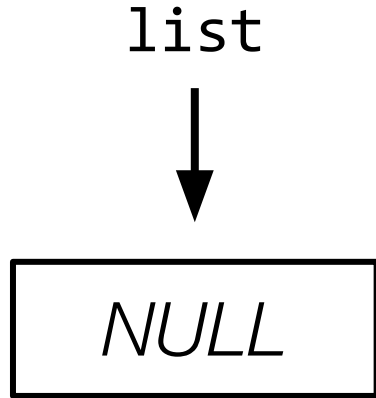
```
node *n = malloc(sizeof(node));
```



```
node *n = malloc(sizeof(node));  
n->phrase = "Hi!";
```



```
node *n = malloc(sizeof(node));  
n->phrase = "Hi!";  
n->next = NULL;
```

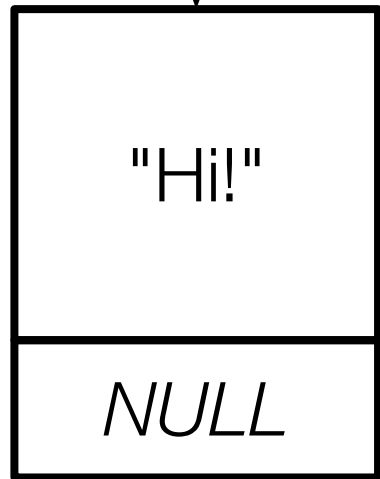


```
list = n;
```

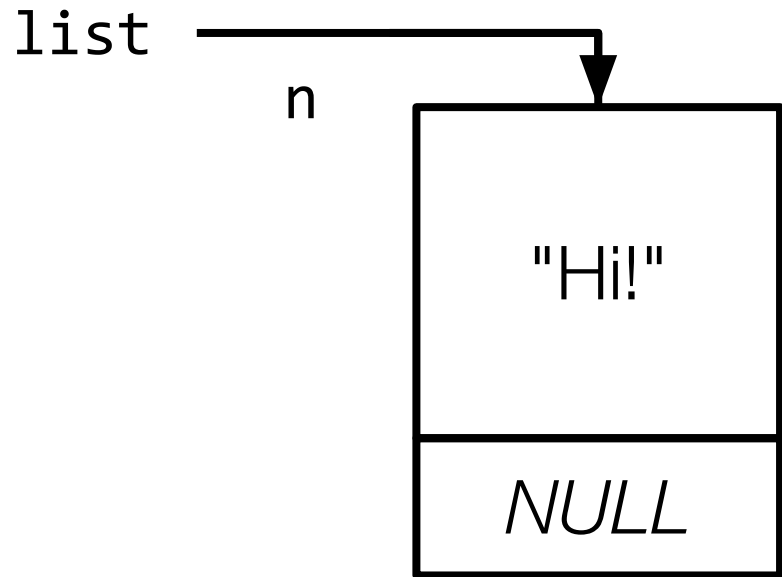
list



n



```
list = n;
```



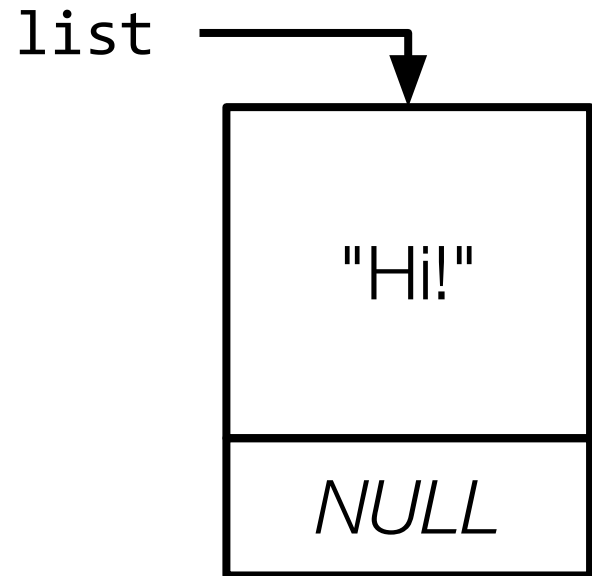
Create a Linked List

Write a program, **list.c**, that creates a linked list of 3 numbers.

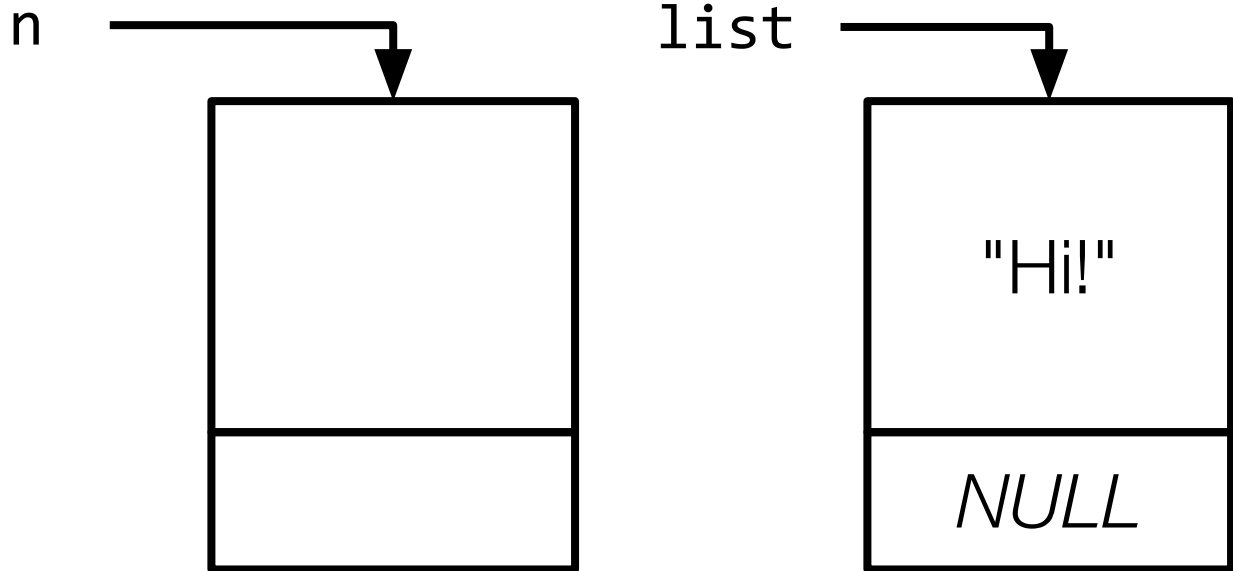
Define a struct **node** storing an integer and a pointer to the next node. In **main**, prompt the user for 3 numbers and dynamically allocate memory for a new node each time.

Inserting Nodes

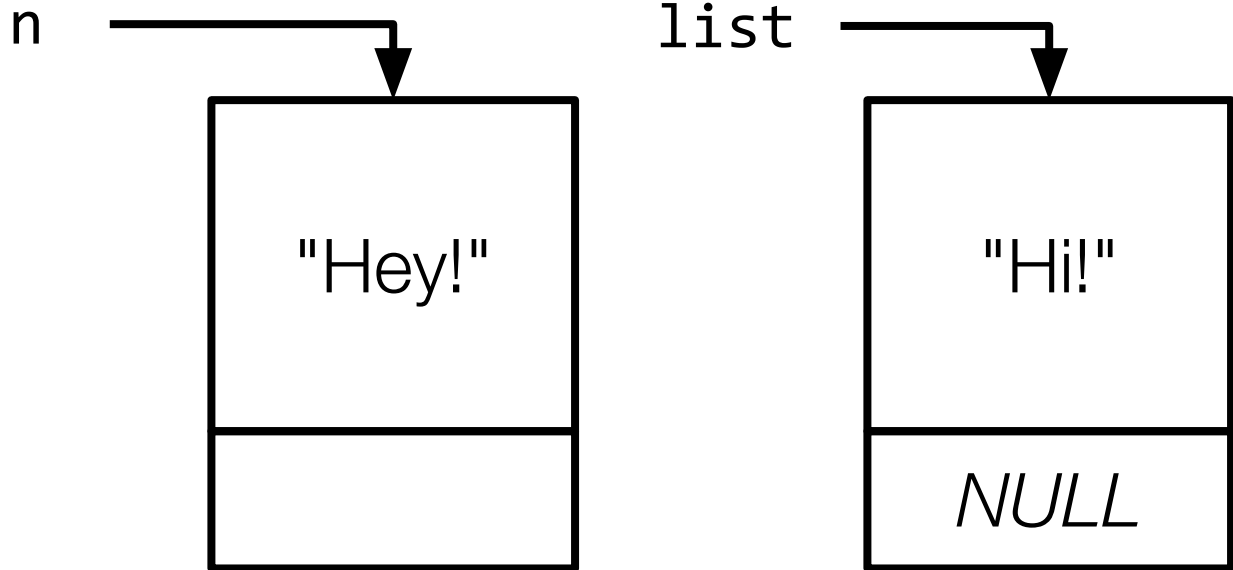
```
n = malloc(sizeof(node));
```



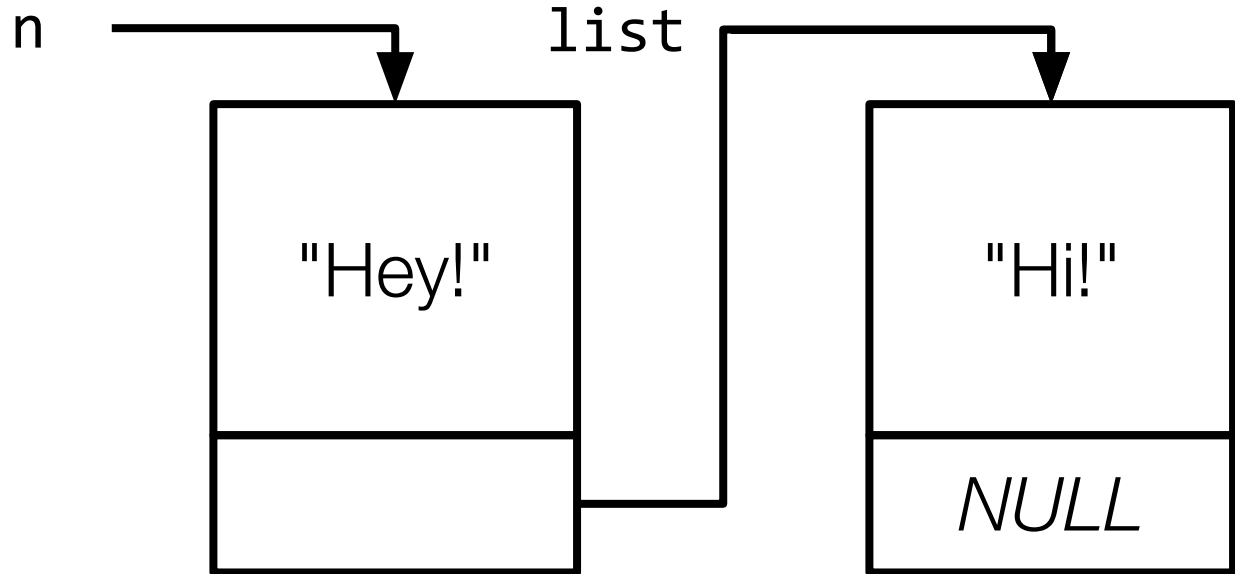
```
n = malloc(sizeof(node));
```



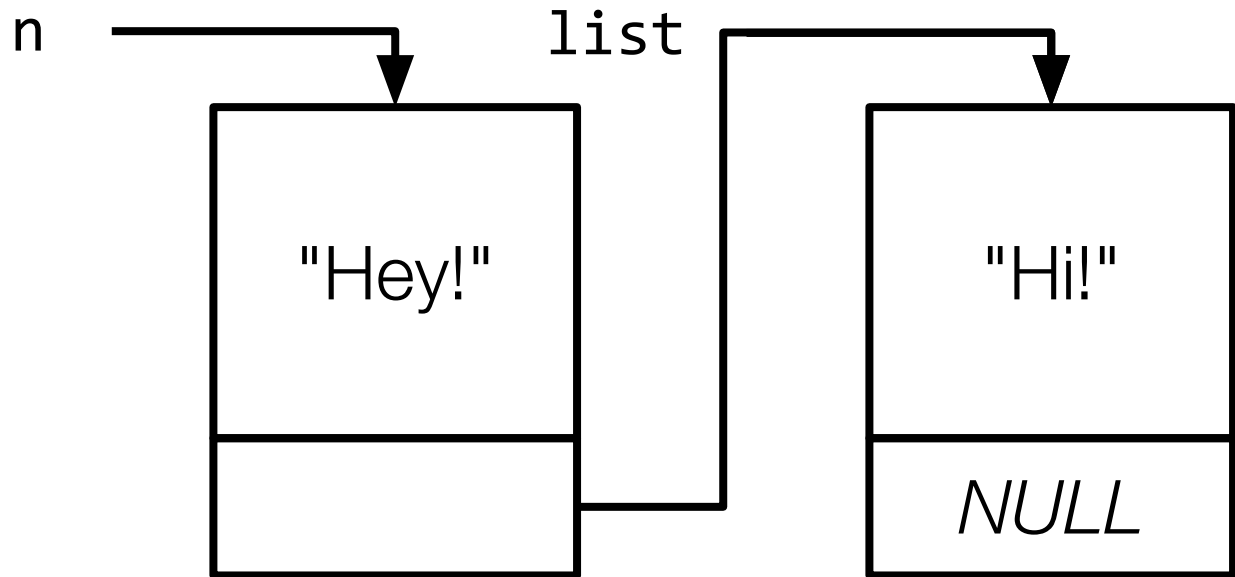
```
n = malloc(sizeof(node));  
n->phrase = "Hey!";
```



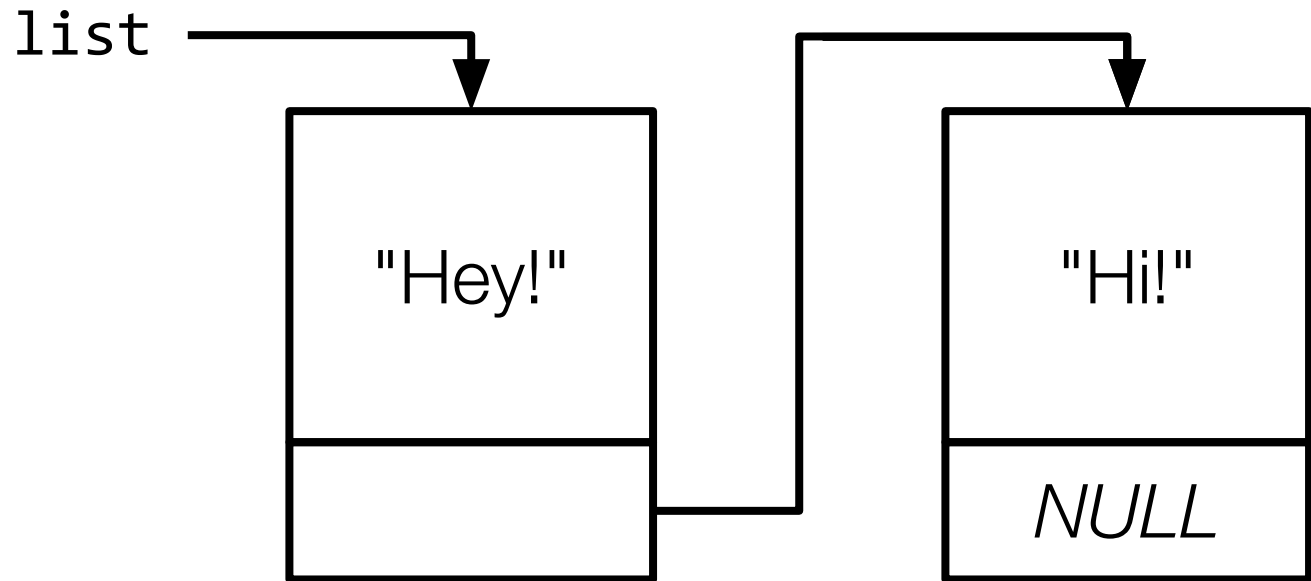
```
n = malloc(sizeof(node));  
n->phrase = "Hey!";  
n->next = list;
```



```
list = n;
```



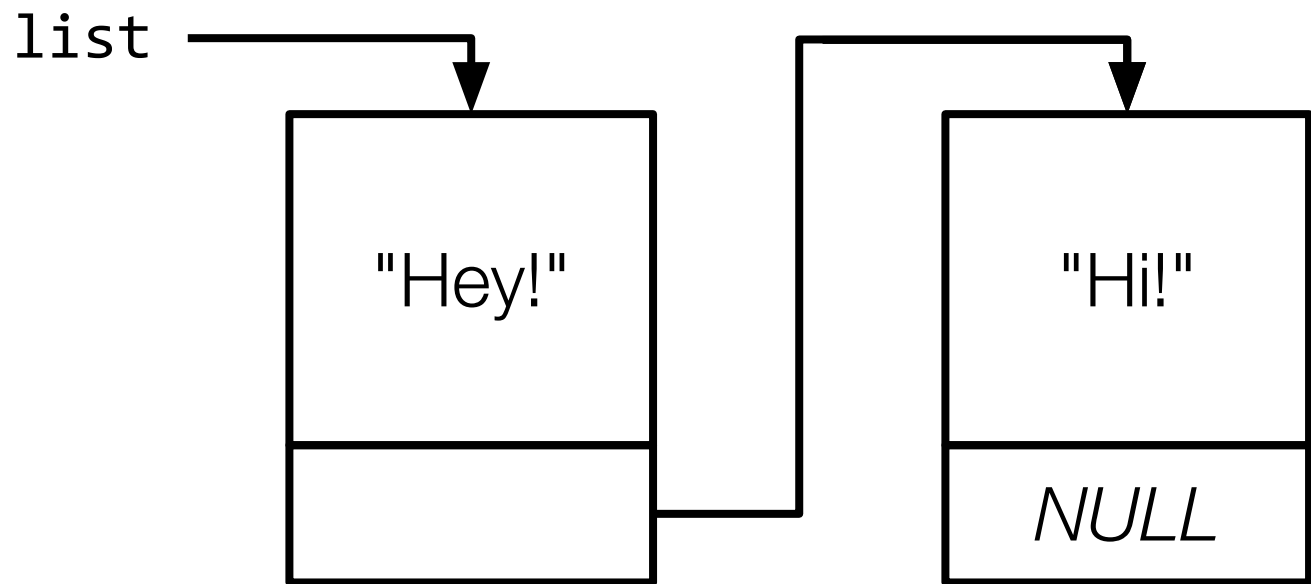
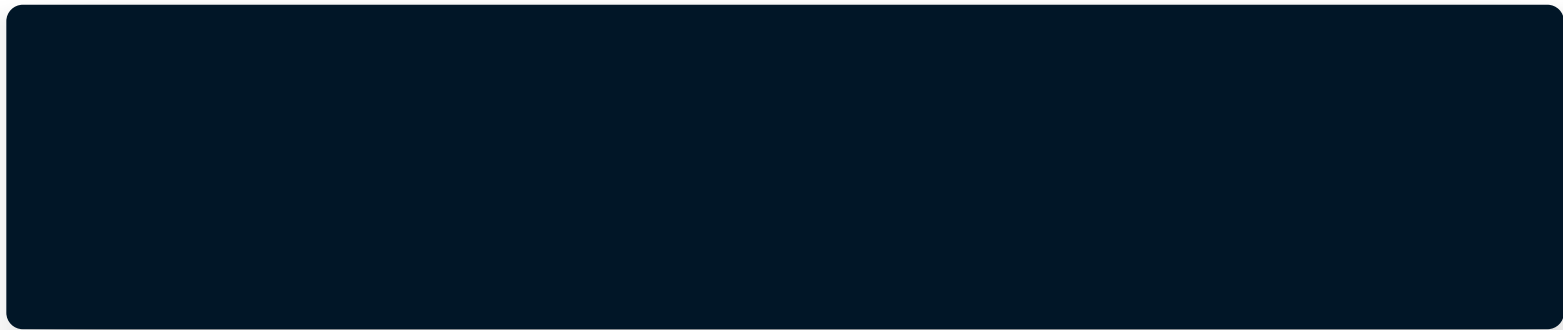
```
list = n;
```



Inserting into a Linked List

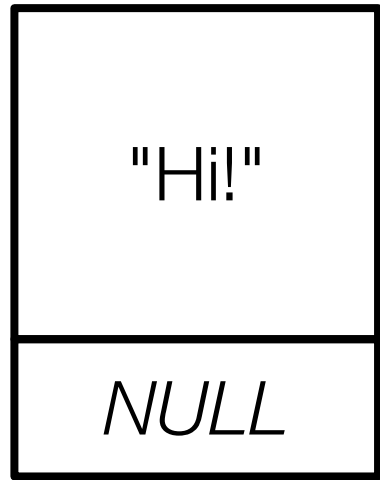
Extend **list.c** so that each new node is inserted at the head of the linked list.

Freeing Nodes



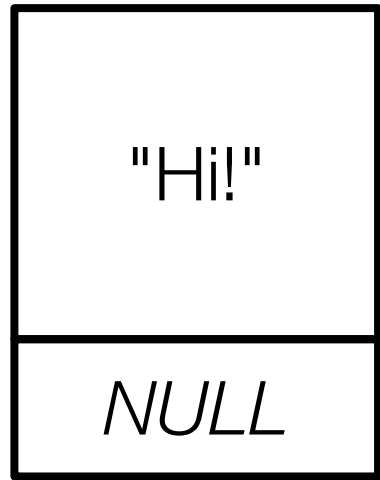
```
free(list);
```

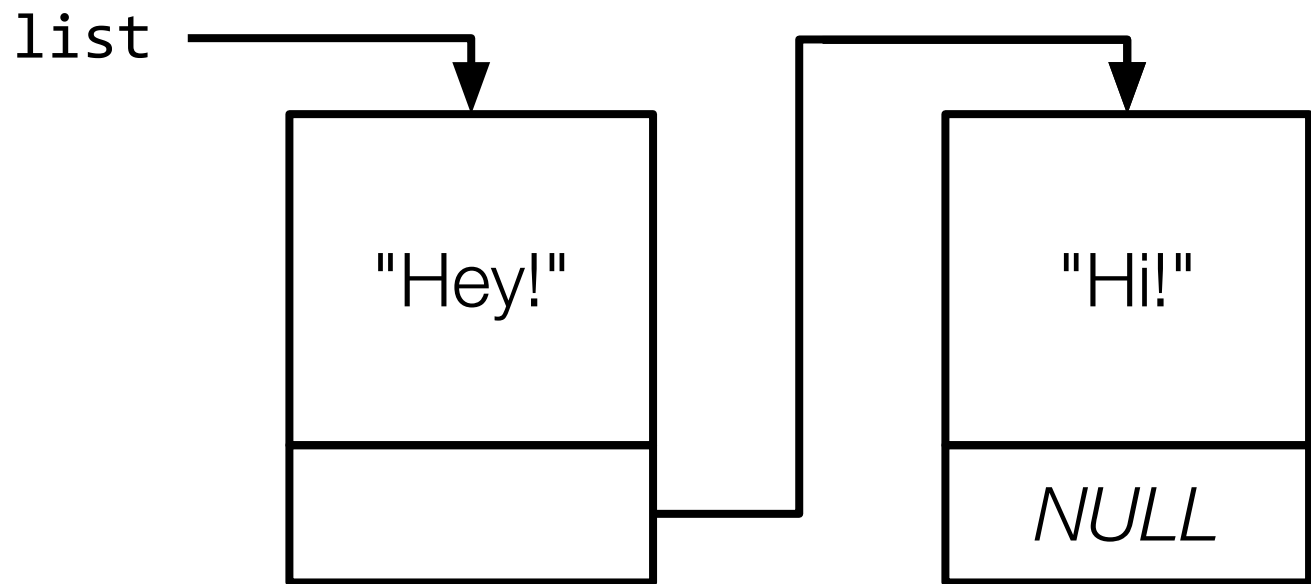
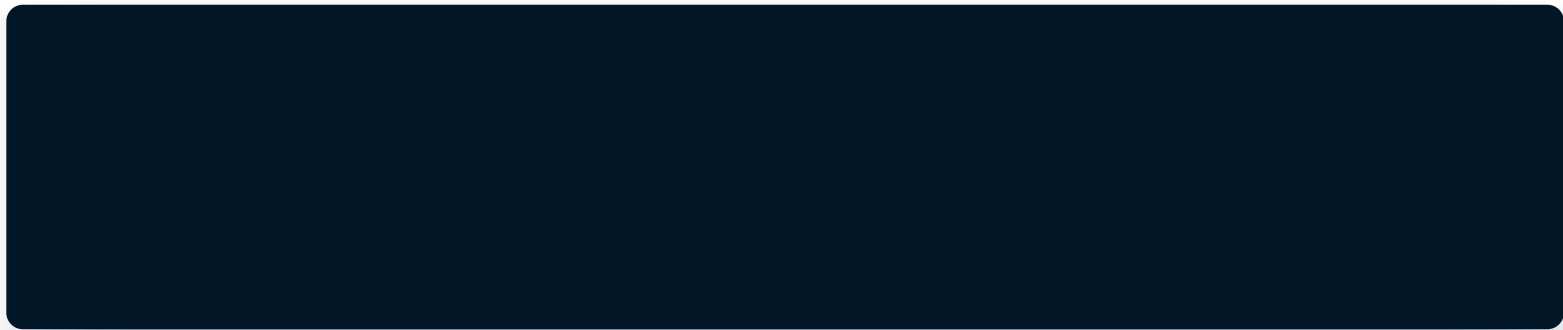
list



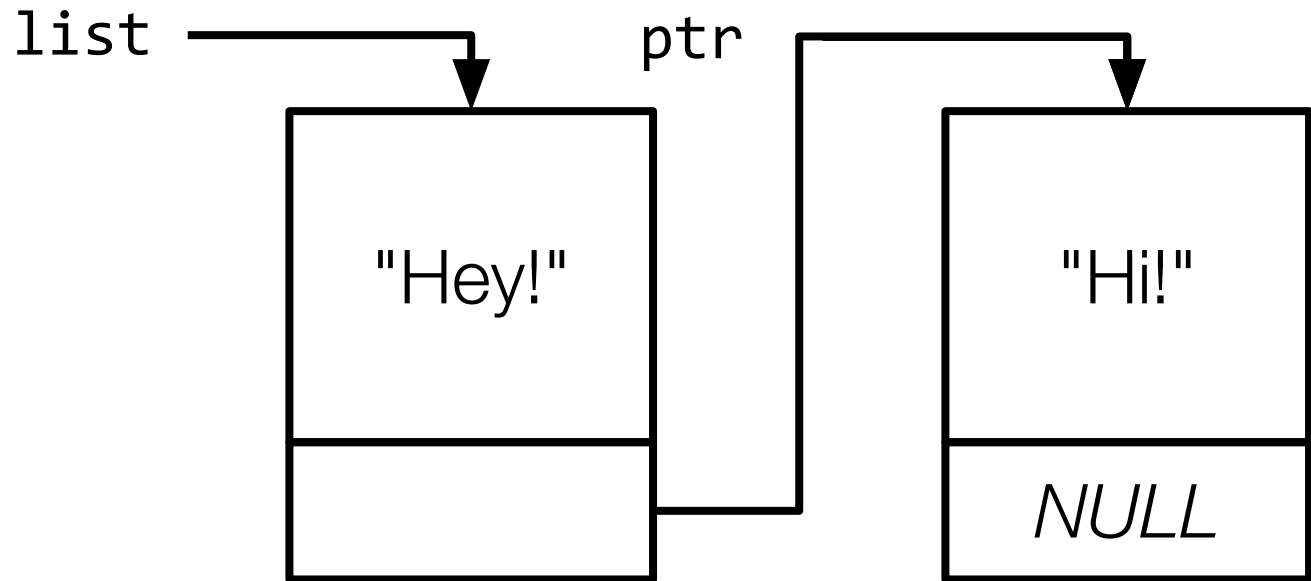
```
free(list);
```

list

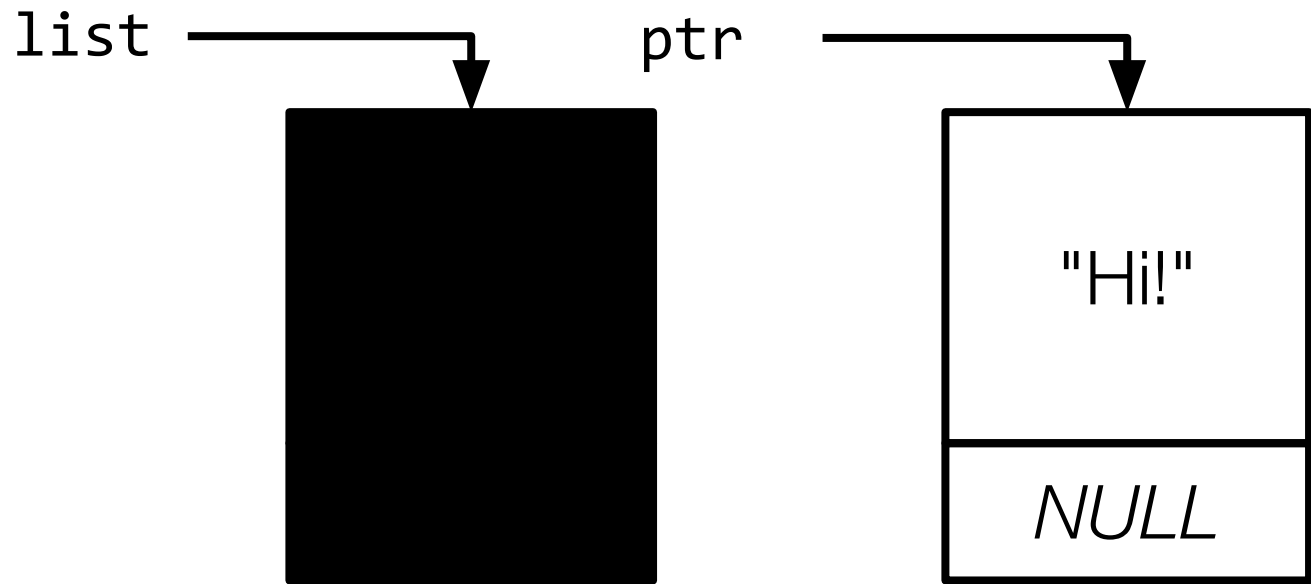




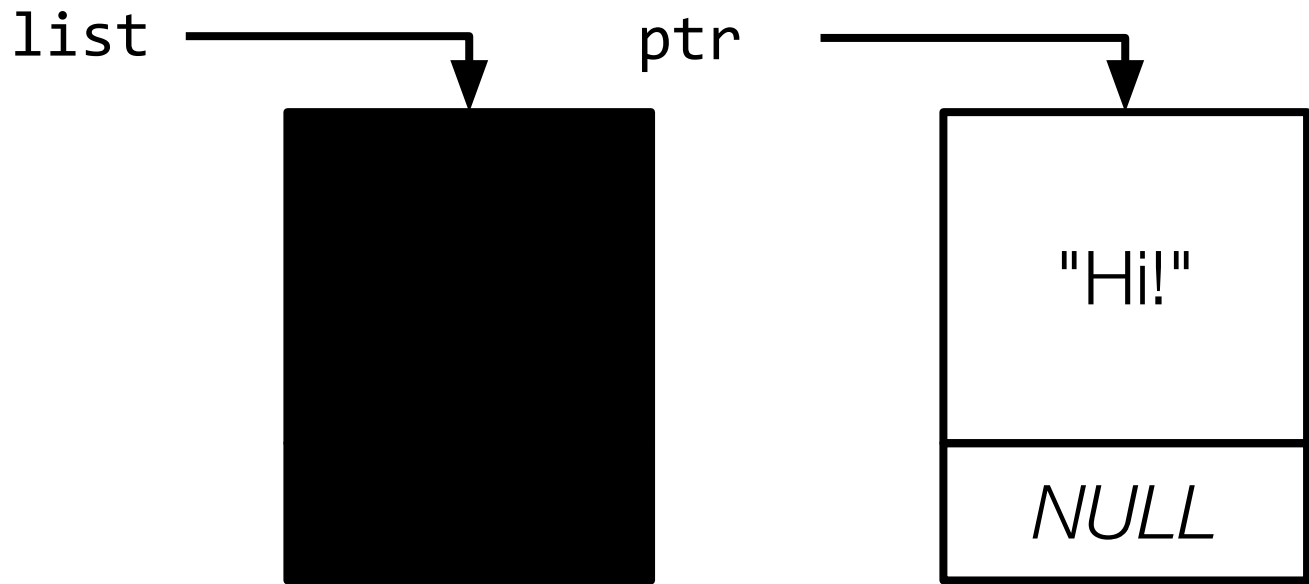
```
node *ptr = list->next;
```



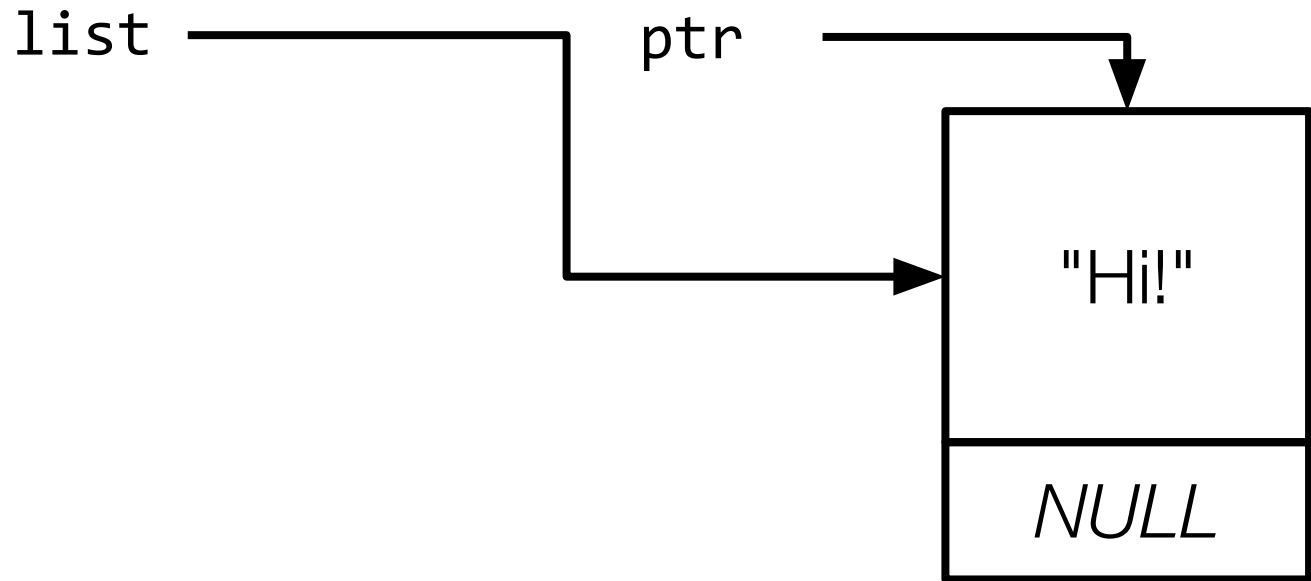
```
free(list);
```



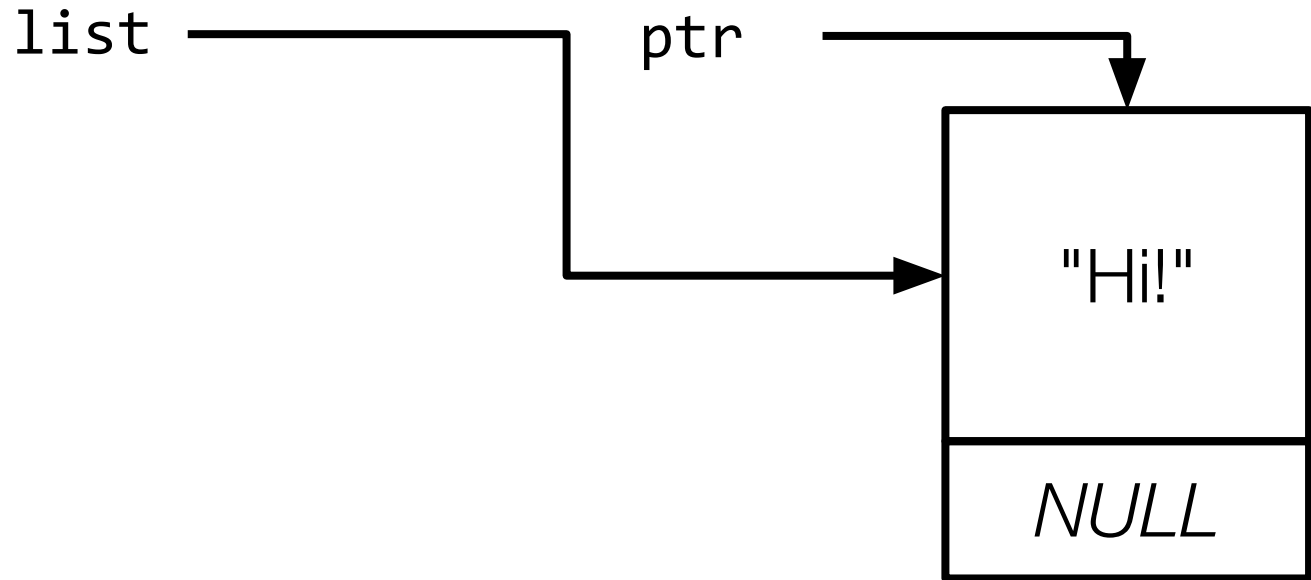
```
list = ptr;
```



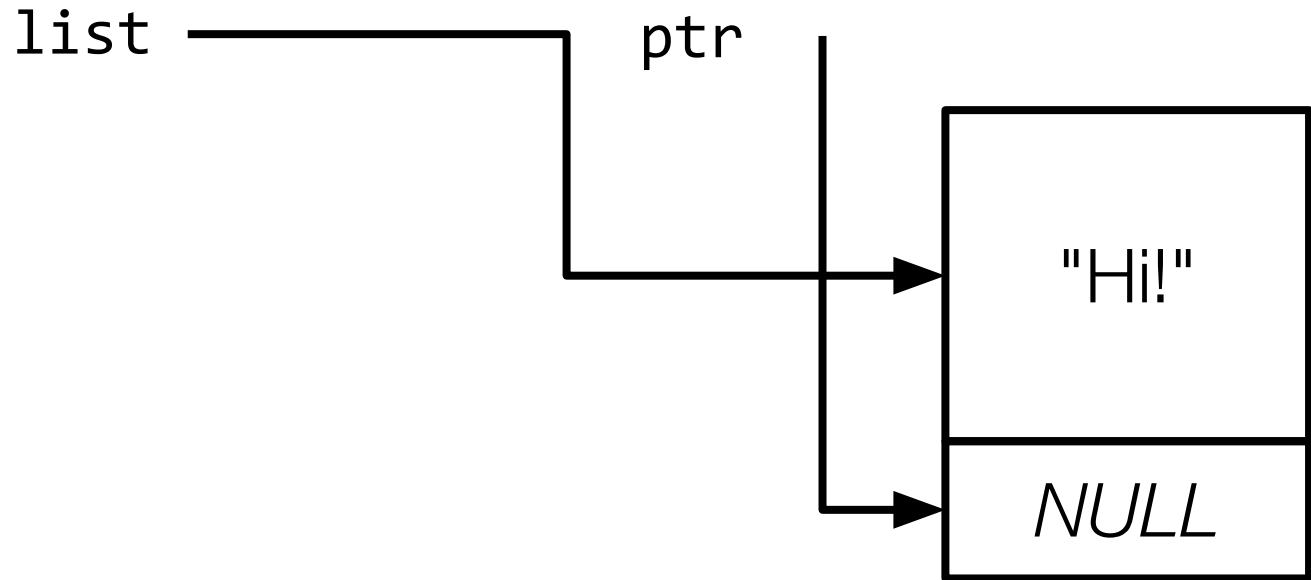
```
list = ptr;
```



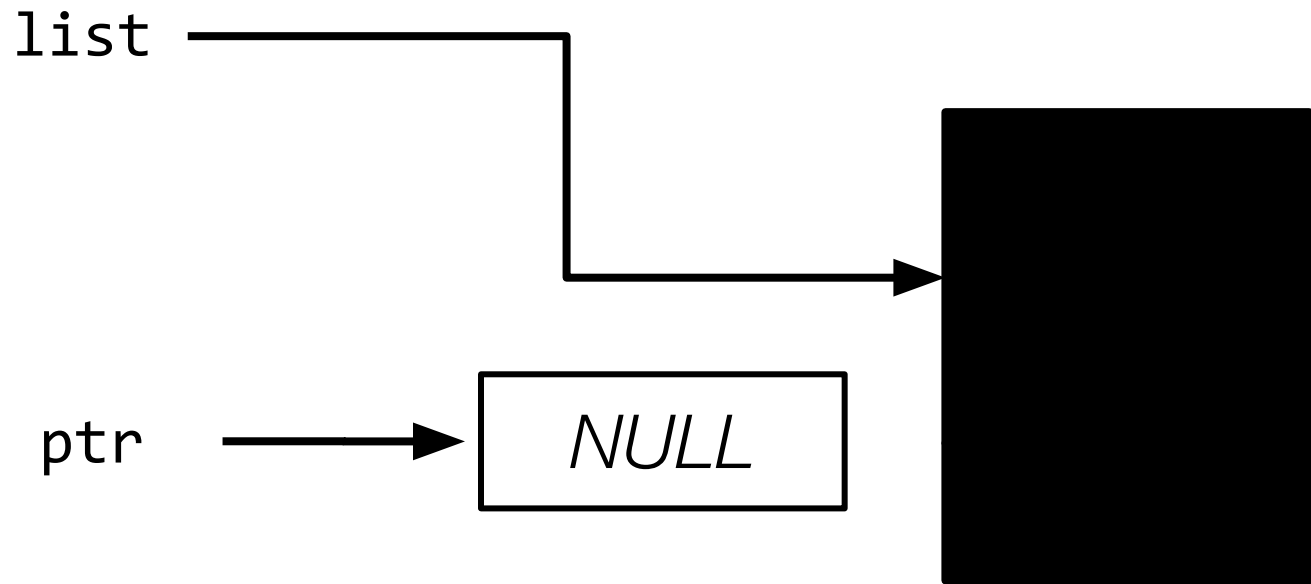
```
ptr = list->next;
```



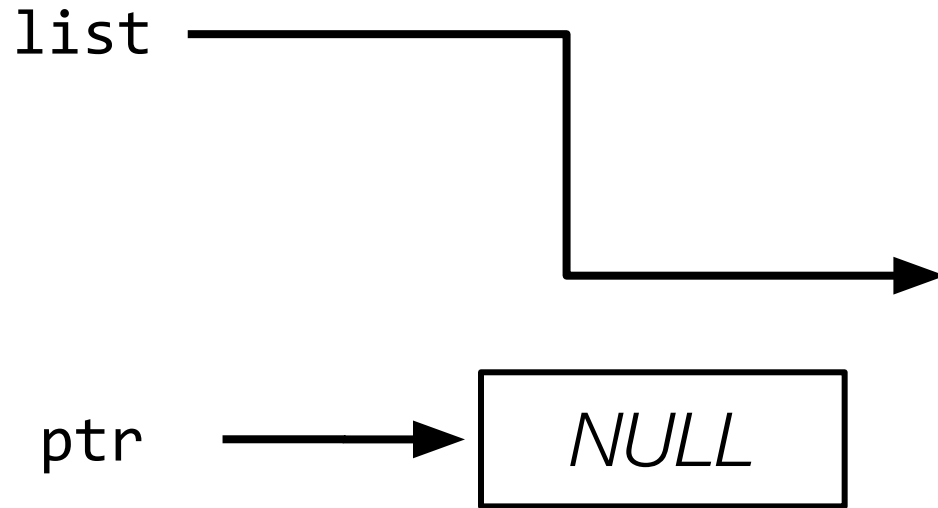
```
ptr = list->next;
```



```
free(list);
```



```
list = ptr;
```



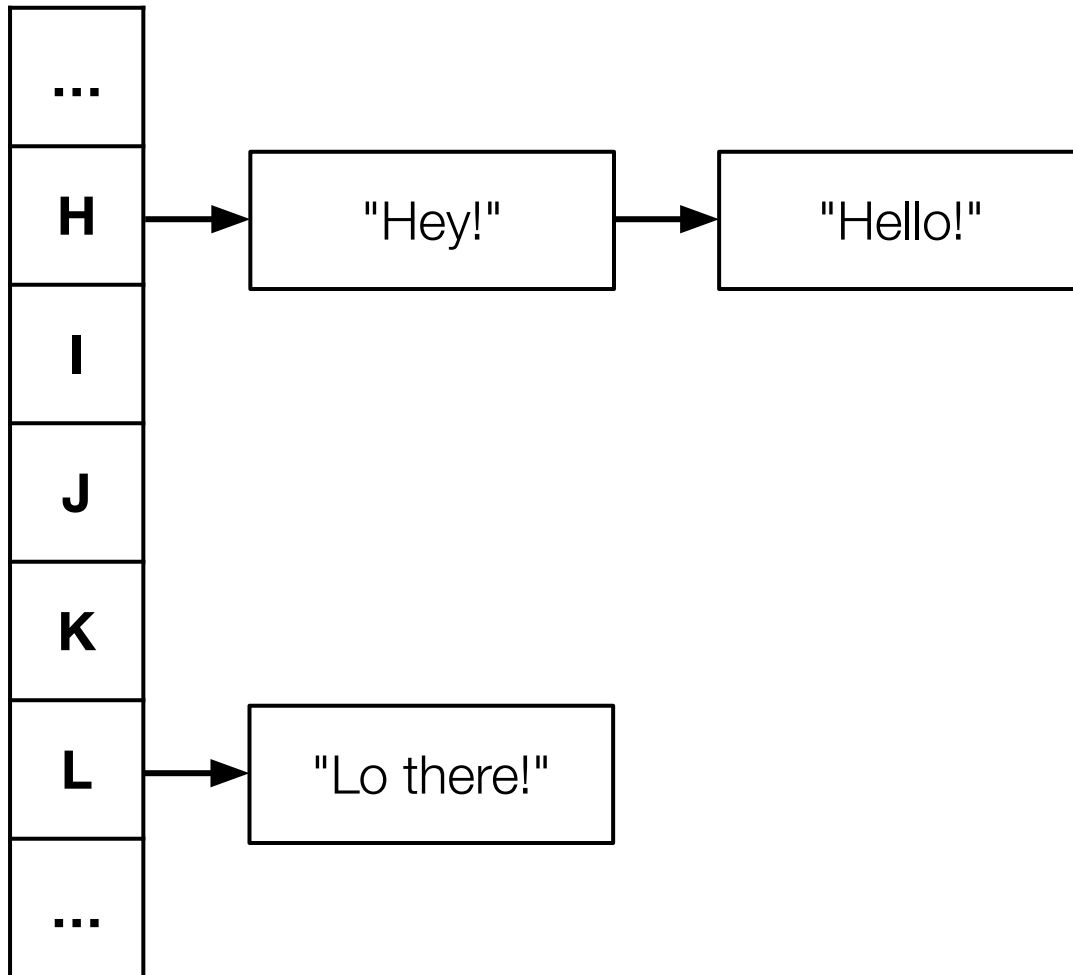
Traversing and Freeing

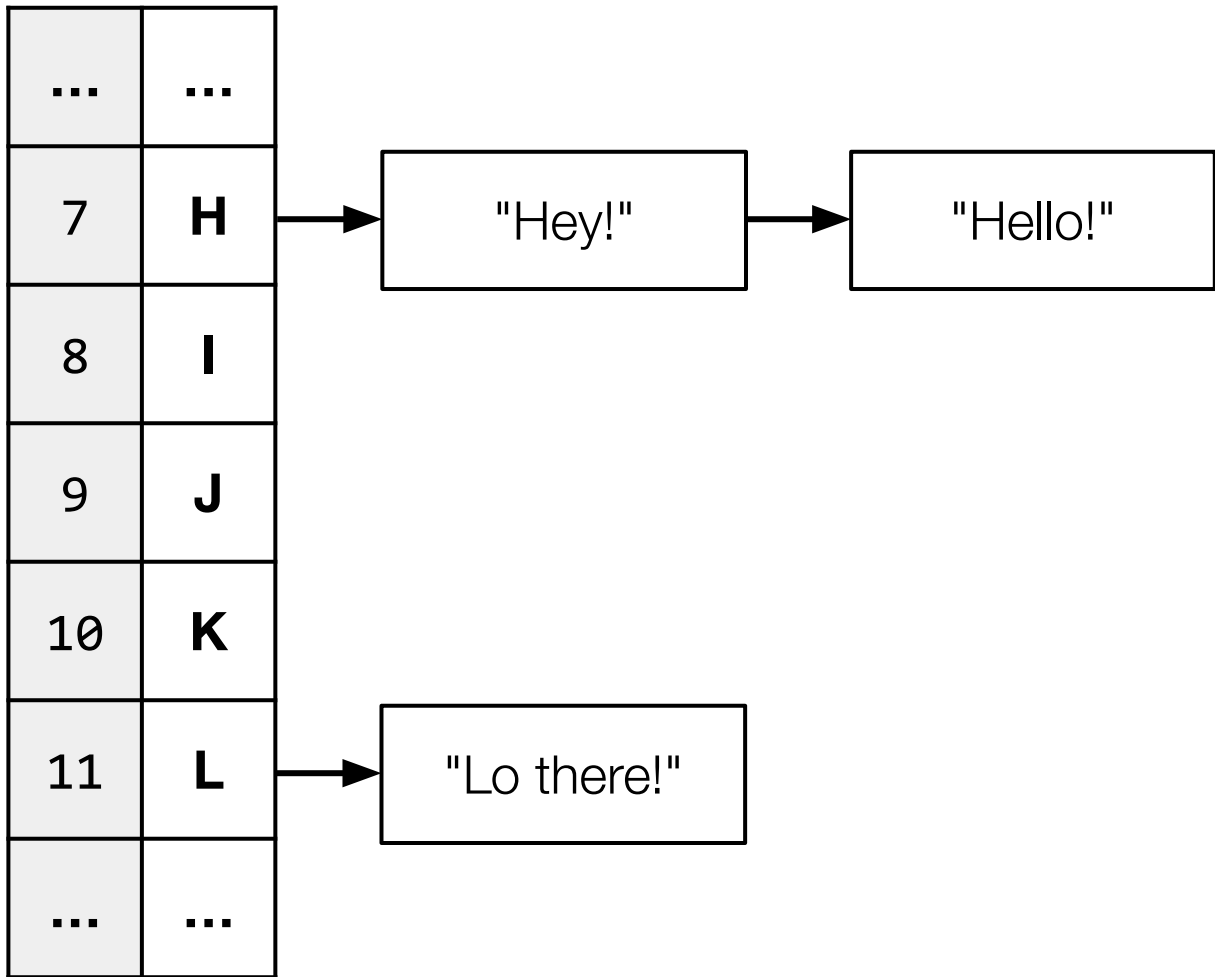
Extend **list.c** to traverse the linked list and print each number. Then, free each node.

Part 2

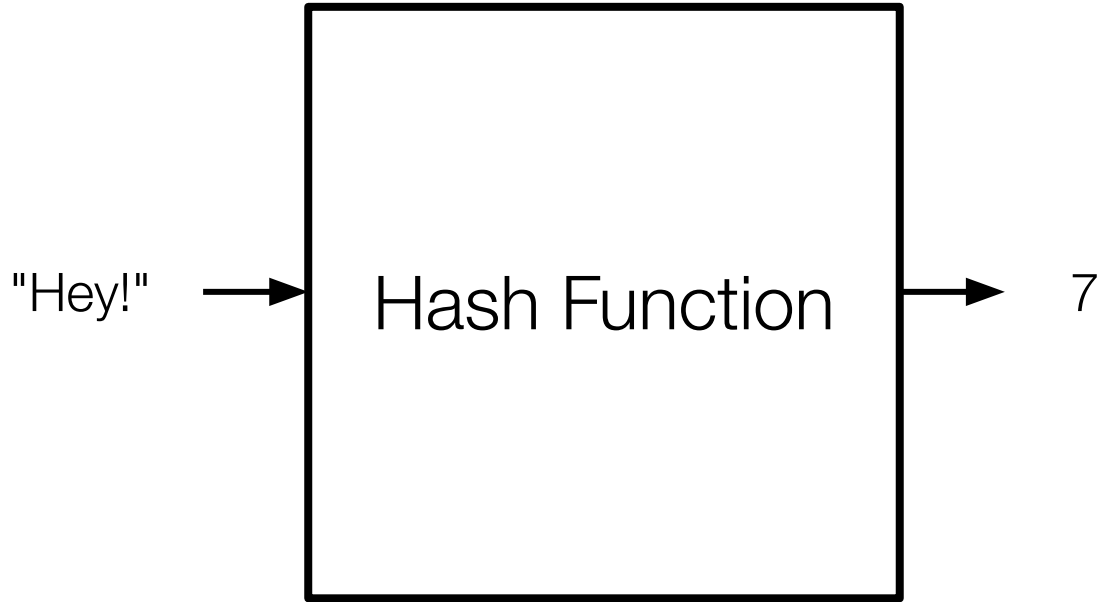
Hash Tables

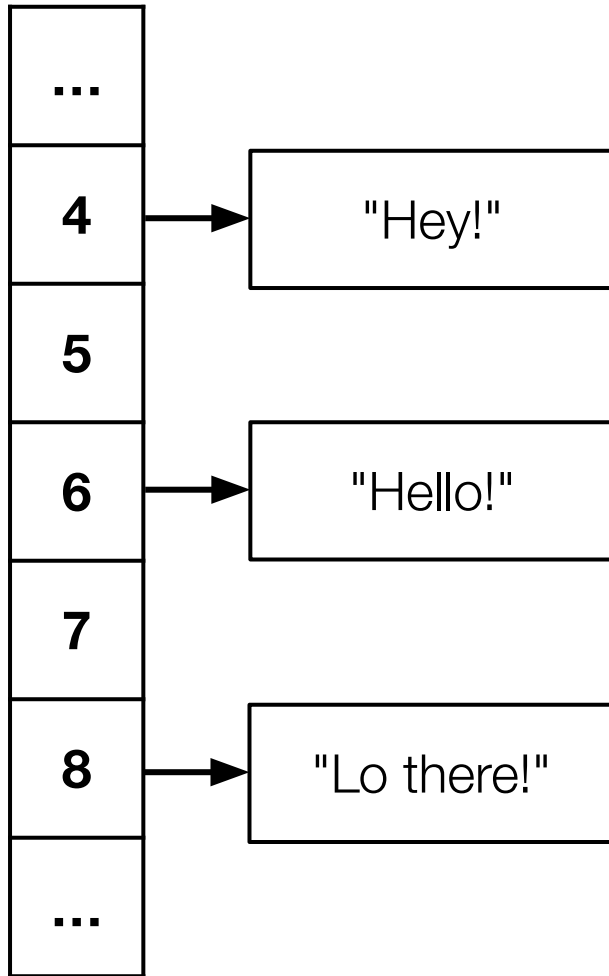






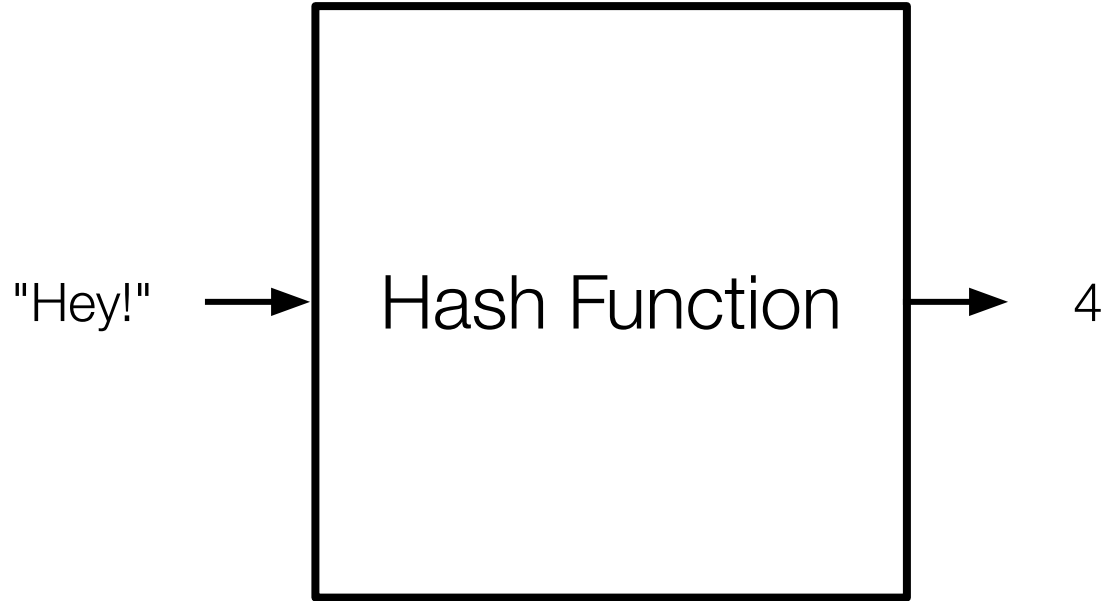
What's the mapping here?





What if we try a different hash function?

What's the mapping here?



A good hash function...

- Always gives you the same value for the same input
- Produces an even distribution across buckets
- Uses all buckets

Are our two examples
good hash functions?

Hashing Exercise

Create a program, **hash.c**, where we will complete a hash function to return a number, 0–25, depending on the first character in the word.

How might we improve this hash function?

This is CS50

Week 5